

EYE GAZE CONTROL MOUSE COMPUTER CURSOR

Varad Naik, David Fernandes, Aaron Gonsalves, Abhijeet Singh, Member, IEEE

ABSTRACT — The Eye Gaze Control Mouse Computer Cursor helps provide access of computer for people with motor disabilities. Low cost and low complexity techniques are used in our technique. The goal of every eye tracking system is to simply determine what it is you're looking at. It needs to be accurate enough to work with a traditional computing workspace, and quick enough to keep up with the fast movement of your eyes. In this system we use hardware such as two Camera to track the display monitor and moment of the reflection of the eye pupil respectively and Microcontroller. Some of the features are: 1) Dense optical flow: which compares previous vision and current vision that shows movement in different parts of the frame 2) Shape detection 3) Optical distance measurement. Microcontroller is based on data acquisition (DAQ) system. In this system we are using Infra-red sensors (IR sensors) for building light-reflection based system with non-imaging sensors like photo-diodes. This system is basically low Complexity system. IR sensors give us accurate approximation of the point of eye gaze. A system can be developed that would listen to DAQ device and performs major tasks like providing a user interface, determining current action performed by user and accordingly performing the action related to the current event. And hence cursor movement can be done. This proposed system will provide a promising interface between computer and humans.

INTRODUCTION

The Eye Gaze Control Mouse Computer Cursor project aims to develop a low-cost, low-complexity system that allows individuals with disabilities to control a computer cursor using their eye movements. This innovative solution leverages existing technologies to create a user-friendly interface, enhancing accessibility and independence for those with physical limitations.

This project focuses on eye tracking technology, which involves determining the user's gaze direction accurately and swiftly. The system employs hardware components such as cameras, microcontrollers, microprocessors, and IR sensors to track eye movements and translate them into cursor actions on a computer screen. Key features include dense optical flow, shape detection, and optical distance measurement, ensuring precise and reliable cursor control. By providing a promising interface between humans and computers, this system aims to enhance the quality of life for people with disabilities.

1.1 Motivation

The Eye Gaze Control Mouse project was inspired by the need to create an accessible and affordable solution for individuals with physical disabilities, allowing them to control a computer cursor using only their eye movements. This approach aims to enhance the independence and quality of life for those who face significant challenges in using traditional input devices such as mice and keyboards.

Existing systems for eye gaze control often come with high costs and complex setups, making them less accessible to the general population. This project addresses these gaps by focusing on a low-cost, low-complexity technique that leverages hardware like cameras, microcontrollers, and IR sensors. The system's features, including dense optical flow, shape detection, and optical distance measurement, provide a reliable and efficient interface. By reducing costs and complexity, this project aims to democratize access to eye-tracking technology, making it available to a broader range of users in need.

1.2 Problem statement

Individuals with hand motor disabilities face significant challenges in using traditional computer mice, limiting their ability to interact with computers effectively. This limitation hinders their access to essential digital tools and resources, reducing their independence and productivity. To address this, we propose the development of an eye-gaze tracking system that allows users to control a computer using only their eye movements. The system will leverage dual cameras to monitor the display and accurately track the reflections of the user's eye pupils. The primary objective is to ensure the system is precise enough to interact seamlessly within a traditional computing workspace and responsive enough to keep pace with the rapid motion of human eyes. By achieving these goals, the solution aims to provide an accessible, efficient, and low-cost alternative to conventional input devices, empowering individuals with motor disabilities to regain control over their digital interactions.

1.3 Objectives

To develop an affordable, user-friendly, and accessible eye-gaze tracking device that empowers both individuals with disabilities and the general population. The device will focus on enhanced optimization and seamless integration across mobile and computer systems, ensuring a versatile and efficient solution for everyday use

METHODOLOGY

3.1 Approach to Project

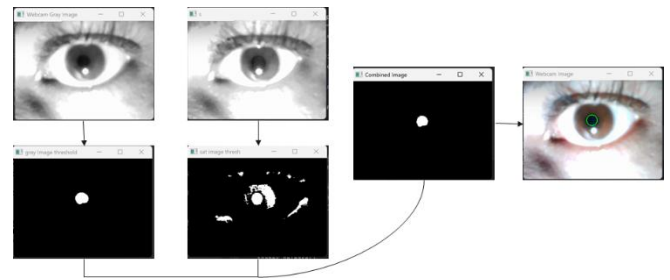


Fig 2

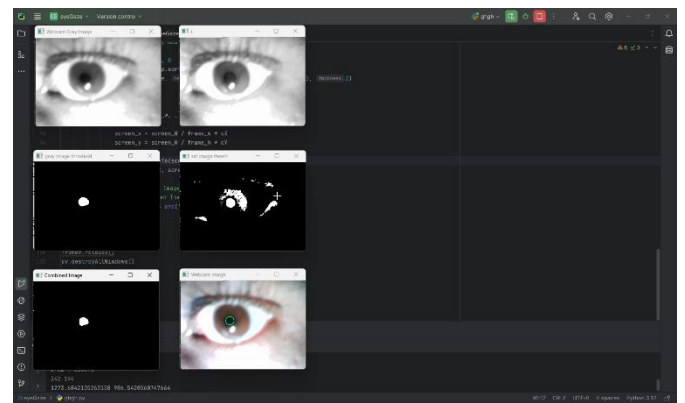


Fig 1

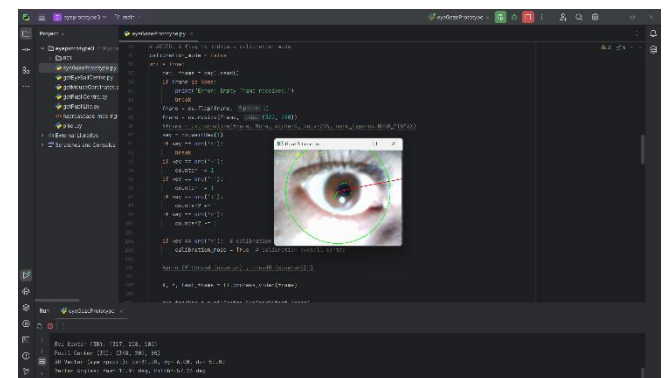


Fig 3

Image Acquisition and Filtering:

Eye-tracking systems capture infrared images of the user's eyes. The images are then processed through noise reduction techniques such as Gaussian filtering, median filtering, or morphological operations to eliminate unwanted artifacts like glare or reflections from glasses.

Projects: Common image filtering techniques are implemented to enhance pupil detection across the various systems

Calibration and Mapping:

A crucial step is calibration, where the system learns to map the user's eye movement to corresponding locations on a computer screen.

Projects: The calibration process is fundamental in systems like those used for eye typing or mouse control to ensure precise cursor movement

3.2 Algorithms

1. eyeGazePrototype.py

Algorithm Description:

This script serves as the main entry point for the Eye Gaze Control Mouse application. It integrates functions to track eye movement and translate it into mouse cursor movements. It combines real-time video capture with gaze tracking algorithms to create an intuitive user experience for individuals with limited mobility. The script plays a pivotal role in managing the overall workflow, ensuring that all components communicate effectively, and presenting a user-friendly interface.

Steps:

Load the necessary libraries (e.g., OpenCV, NumPy).

Initialize the video capture to get live video feed from the camera.

Detect eyes using a pre-trained Haar cascade classifier to identify the eye region and refine the detection through iterative adjustments.

Call sub-functions from other modules (e.g., getPupilLite, getMouseCoordinates) to

process the eye gaze and control the mouse cursor.

Continuously loop until the application is terminated, processing frames in real time and handling exceptions such as unexpected input or camera errors.

Provide visual feedback by displaying the processed video feed with overlays indicating detected gaze points.

Log important events and errors to assist in debugging and improving robustness.

Pseudocode:

1. Start
2. Initialize video feed and load necessary resources
3. while feed is active:
4. Capture frame from camera
5. Detect eye region using Haar cascade classifier
6. Identify pupil location using getPupilLite
7. Map gaze coordinates to mouse position using getMouseCoordinates
8. Move mouse to the calculated position on screen
9. Display real-time feedback overlay on video feed
10. Handle exceptions and log errors
11. End

2. getEyeBallCentre.py

Algorithm Description:

Detects the center of the eyeball within the eye region using image processing techniques. This module plays a crucial role in accurately determining gaze direction by identifying the center of the pupil and ensuring consistent tracking. It employs robust methods to handle variations in

lighting, noise, and eye movements, contributing to high accuracy.

Uses image processing techniques like grayscale conversion, adaptive thresholding, and contour detection to isolate the region of interest effectively.

Steps:

Convert the input eye region image to grayscale to simplify processing and reduce computational overhead.

Apply adaptive thresholding to enhance contrast between the pupil and surrounding area, ensuring reliable detection across varying lighting conditions.

Identify contours to locate the pupil region accurately, filtering out irrelevant contours based on size and shape.

Calculate the center of the detected contour to determine the eyeball center, ensuring the results are robust to noise and partial occlusions.

Return the center coordinates for integration with other modules and provide confidence metrics for additional validation.

Flowchart:

Input: Eye region image.

Process: Grayscale → Adaptive Threshold → Contours → Center Calculation.

Output: Coordinates of the eyeball center.

3. getMouseCoordinates.py

Algorithm Description:

Maps the detected gaze position to screen coordinates with high accuracy. This mapping ensures that the user's gaze is translated to precise mouse movements across the screen, enabling smooth control. The module is adaptable to various screen sizes and resolutions, making it versatile for different user setups.

Scales the relative position of the gaze within the camera feed to the screen resolution to accommodate varying screen sizes and camera placements.

Steps:

Receive gaze coordinates as input from the detection module.

Calculate the relative position of the gaze point within the frame by normalizing the coordinates to the frame dimensions.

Scale the relative position to the screen resolution, ensuring proportional mapping regardless of device or screen orientation.

Implement smoothing algorithms to reduce jitter and ensure stable cursor movement.

Return the mapped coordinates to be used by the mouse control function and log the results for further analysis.

Pseudocode:

1. Input: Gaze coordinates (x, y), Frame size (width, height), Screen resolution (sw, sh)
2. $RelativeX = x / width$
3. $RelativeY = y / height$
4. $MouseX = RelativeX * sw$
5. $MouseY = RelativeY * sh$
6. Apply smoothing to MouseX, MouseY
7. Return MouseX, MouseY

4. getPupilLite.py

Algorithm Description: The `getPupilLite` module is a lightweight function designed to detect the pupil's location in an optimized and efficient manner. By reducing computational overhead, it achieves real-time performance while maintaining accuracy. This makes it particularly suitable for continuous gaze tracking applications.

Unlike `getEyeBallCentre`, which is more comprehensive but resource-intensive, `getPupilLite` focuses on computational simplicity. It uses streamlined methods for contour detection and thresholding while preserving enough precision for reliable gaze tracking. This module is specifically tailored for environments requiring rapid processing and minimal latency.

Steps

Input Preprocessing: The input image, which represents the eye region, is resized to a lower resolution. This resizing ensures faster processing by reducing the amount of data to analyze without compromising the critical details needed for pupil detection. Noise reduction is applied using filters like **Gaussian blur** or **median filters** to eliminate artifacts from the image. These filters smoothen the image, enhancing the visibility of the pupil by reducing distractions caused by noise or uneven lighting.

Thresholding: Thresholding is a method to convert the grayscale image into a binary image, where the pixel values are either 0 or 255. This process helps in isolating the pupil (the darkest region) from the surrounding eye features.

In this module, a simplified global thresholding method is primarily used. A fixed threshold value is selected, setting all pixel intensities below this value (representing darker regions) to 0 (black) and all others to 255 (white). This approach is computationally efficient and works well in environments with consistent lighting. However, when lighting conditions are uneven or variable, **adaptive thresholding** is employed. Adaptive thresholding determines the threshold dynamically for different regions of the image based on local pixel intensities. This method divides the image into small blocks and computes the threshold value for each block, effectively handling variations in illumination and ensuring the pupil remains isolated even in challenging conditions.

The choice of thresholding technique depends on the environmental factors. For consistent performance, the module monitors the overall brightness of the input image and dynamically switches between global and adaptive thresholding as needed. This hybrid approach ensures reliable segmentation in diverse scenarios.

Contour Detection: Contours are extracted from the binary image generated during thresholding. Contours are continuous curves that bound or define the shape of an object in an image.

From the list of detected contours, filtering is applied based on:

Area: Ensuring the contour size falls within an expected range for the pupil.

Circularity: Validating that the shape is approximately circular, as the pupil generally appears round.

Position: Verifying the contour lies within the expected region of the eye.

This filtering ensures only the most likely candidate for the pupil is selected, ignoring irrelevant objects like reflections or noise.

Validation: The detected pupil's characteristics, such as size, shape, and position, are cross-verified with expected parameters to ensure accuracy.

Additional consistency checks may include verifying the position relative to other facial landmarks, like the eye corners or iris boundary, to confirm it aligns with the expected anatomical structure.

Detections failing these criteria are discarded to prevent false positives, ensuring reliability.

Output Coordination: Once the pupil coordinates are validated, they are returned to the calling module. These coordinates serve as the critical input for downstream tasks like gaze mapping.

Simultaneously, feedback mechanisms adjust the thresholding or preprocessing parameters dynamically based on observed

conditions (e.g., varying lighting or user movement).

Integration: The detected pupil location is passed to higher-level modules, such as `getMouseCoordinates`, ensuring alignment with the entire pipeline for gaze tracking and mouse control.

Logs of the detection process are maintained, capturing metrics such as processing time, success rates, and any anomalies. These logs assist in debugging and continuous improvement of the module.

Block diagram

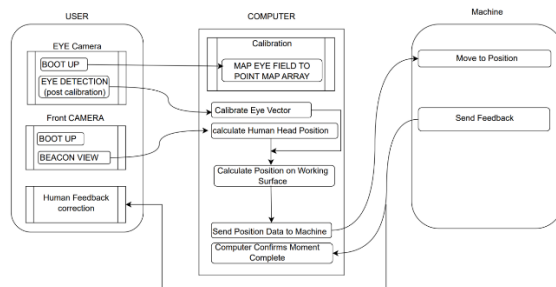


Fig 4

4.2 Detail flow of Architecture

Purpose: This architecture focuses on detecting eye movements, identifying the pupil, and controlling the mouse cursor based on gaze. It is designed to provide an intuitive, real-time interaction experience for users, especially those with mobility challenges. The system prioritizes accuracy, adaptability, and user convenience through modular design and robust algorithms.

Components:

Video Input: Captures live feed from the camera, serving as the foundation for gaze tracking. The input is preprocessed to optimize quality and frame rate.

Image Processing: Processes frames to detect eyes and pupils, utilizing advanced

computer vision techniques, including machine learning where applicable.

Gaze Mapping: Maps pupil position to mouse coordinates, ensuring accurate translation of gaze to screen, and supports dynamic adjustments to calibration.

Mouse Control: Moves the mouse cursor to the mapped location, allowing users to interact with the system seamlessly. Provides options for click emulation and gesture-based actions.

Component Interactions

1. `eyeGazePrototype.py`:

Acts as the central coordinator, managing the workflow of the entire application.

Interacts with `getPupilLite.py` to identify the pupil location and `getMouseCoordinates.py` to calculate the cursor position.

Displays feedback to the user to enhance usability and debug issues in real time. Logs critical metrics and errors for analysis.

2. `getEyeBallCentre.py`:

Processes the eye region image to identify the eyeball center.

Provides critical data for more advanced gaze

calculations and contributes to higher accuracy in cursor movement. Implements fallback mechanisms for noisy input.

3. `getMouseCoordinates.py`:

Maps gaze to screen coordinates, translating input from other modules into actionable mouse movements. Incorporates smoothing algorithms to reduce jitter and enhance stability.

Feeds final cursor position back to `eyeGazePrototype.py` to ensure smooth interaction and logs mapped coordinates for review.

4. **getPupilLite.py:**

Lightweight and efficient module for pupil detection, balancing accuracy with computational speed. Adapts dynamically to changes in input quality and lighting conditions.

Diagrams and Tables

Architectural Block Diagram:

Input: Live video feed.

Processing: Eye detection → Pupil detection → Gaze mapping.

Output: Cursor movement.

Component	Responsibility
<code>eyeGazePrototype.py</code>	Overall coordination and display.
<code>getEyeBallCentre.py</code>	Eyeball center detection.
<code>getMouseCoordinates.py</code>	Gaze-to-mouse mapping.
<code>getPupilLite.py</code>	Pupil detection.

the potential for widespread adoption due to its affordability and ease of use. The system's key features, including dense optical flow, shape detection, and optical distance measurement, ensure precise and reliable performance, making it a promising solution for enhancing the quality of life for people with disabilities.

Overall, this project highlights the importance of leveraging technology to create inclusive solutions that empower individuals and promote greater independence. The successful implementation of the Eye Gaze Control Mouse's Phase 1 sets a foundation for future advancements in assistive technologies, further contributing to the field of accessibility and human-computer interaction.

REFERENCES

CONCLUSION

The Eye Gaze Control Mouse project has successfully demonstrated a low-cost, low-complexity system that so far, can be used to move a cursor. By integrating hardware such as cameras and microcontrollers, we have developed an accessible and efficient interface that accurately tracks eye gaze and translates it into cursor actions.

This innovative approach not only bridges the gap in existing high-cost and complex eye-tracking systems but also showcases